

MIPS: Estrutura, Aritmética e Entrada/Saída

Aula 3 — Programação e Simulação Prática no MARS

PROGRAM COUNTER (PC)

\$0.0	...
\$0.1	...
\$3.0	...
\$3.1	...
\$3.3	...

ALU (Arithmetic Logic Unit)

\$10	= (\$10)
\$11	= (\$21)
\$12	= (\$32)
\$13	= (\$23)
\$14	= (\$31)
\$15	= (\$23)

DATA MEMORY

A11: 15

INSTRUCTION MEMORY

Sw \$t1, 0(\$sp)
Add \$t03
Swb \$t11



Prof. Danilo Monteiro Souza

.data (O Almoxarifado)



Área destinada à declaração de variáveis e dados estáticos na memória principal (RAM) antes do programa rodar.

```
.word  
.asciiz  
.space
```

.text (A Linha de Produção)



Onde residem as instruções, a lógica e as operações do programa. O processador lê e executa essa área linha por linha.

```
main:  
    add $t0, $s0, $s1
```

O caractere '#' atua como o '//' no C. O montador ignora tudo após ele na linha.

Aritmética: A Regra de Três do MIPS

Syntax Matrix

[OPCODE] [DESTINO], [FONTE 1], [FONTE 2]

add \$t0, \$s1, \$s2

Ação (O que fazer?):
A instrução ou
operação desejada.

Destino (Onde guardar?):
O registrador que receberá
o resultado final.

Fontes (Com o que calcular?):
Os operandos ou valores de
entrada da equação.

A arquitetura MIPS preza pela regularidade. O formato de 3 **operandos** mantém o hardware simples, rápido e padronizado.

Aritmética na Prática (add, sub, addi)

Soma de Registradores

Instrução:
`add $t0, $t1, $t2`

Tradução Humana:
 $\$t0 = \$t1 + \$t2$

Uso:
Soma o conteúdo de duas variáveis.

Subtração

Instrução:
`sub $t0, $t1, $t2`

Tradução Humana:
 $\$t0 = \$t1 - \$t2$

Uso:
Subtrai \$t2 de \$t1.

Soma com Imediato (Constante)

Instrução:
`addi $t0, $t1, 100`

Tradução Humana:
 $\$t0 = \$t1 + 100$

Uso: Soma um valor numérico direto a um registrador.



Atenção: Não existe instrução subi nativa (subtração com imediato) no hardware MIPS. Para subtrair uma constante, utilize a soma com um valor negativo:

`addi $t0, $t1, -50`

Syscalls: O Garçon do Sistema Operacional



Tabela de Syscalls: Print e Read de Inteiros

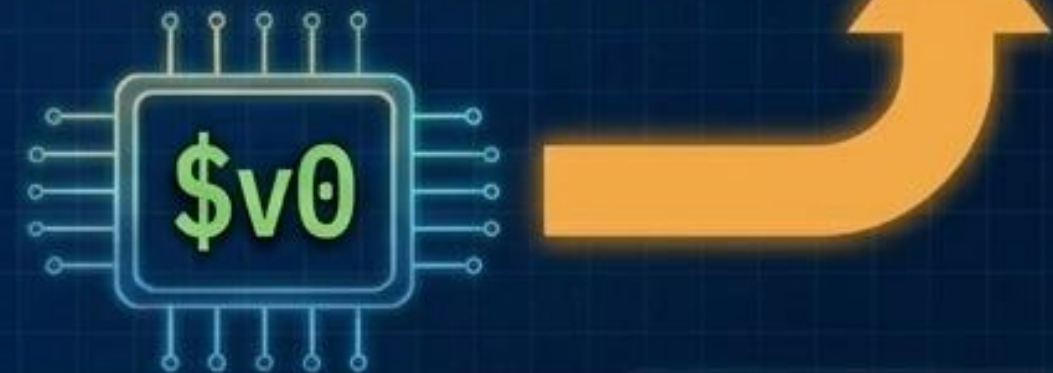
Saída (Print Integer)

- Código do Serviço (\$v0): 1
- Entrada (\$a0): Valor numérico que será impresso na tela.
- Comportamento: O SO pega o valor da bandeja \$a0 e joga no console.



Entrada (Read Integer)

- Código do Serviço (\$v0): 5
- Retorno: O usuário digita o número. O SO salva o valor recebido dentro do próprio \$v0.
- Comportamento: O programa congela até o usuário dar Enter no teclado do simulador MARS.



Estudo de Caso: Soma de Dois Números (1/2)

```
.text  
main:
```

```
    li $v0, 5      # Prepara para ler o 1º inteiro  
    syscall        # Executa a leitura (congela a tela)  
    move $t0, $v0  # Cofre: Salva o número seguro em $t0
```

`li` (Load Immediate) é uma pseudo-instrução para colocar o número 5 diretamente no `$v0`. É o nosso código 5 do menu.

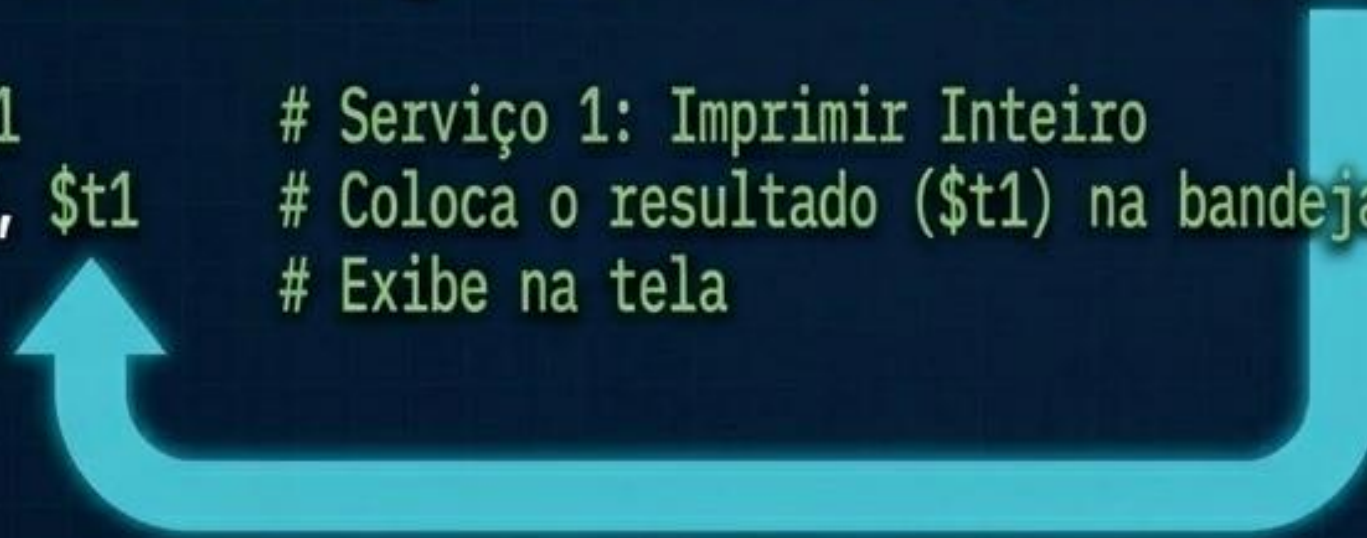
Crucial! O próximo `syscall` vai sobrescrever o `$v0`. Usamos `move` para proteger o primeiro número no registrador temporário `$t0` antes de continuarmos.

Estudo de Caso: Processamento e Exibição (2/2)

```
li $v0, 5      # Prepara para ler o 2º inteiro
syscall        # 0 segundo número entra em $v0

add $t1, $t0, $v0 # Magia: $t1 = $t0 (Num1) + $v0 (Num2)

li $v0, 1      # Serviço 1: Imprimir Inteiro
move $a0, $t1  # Coloca o resultado ($t1) na bandeja ($a0)
syscall        # Exibe na tela
```

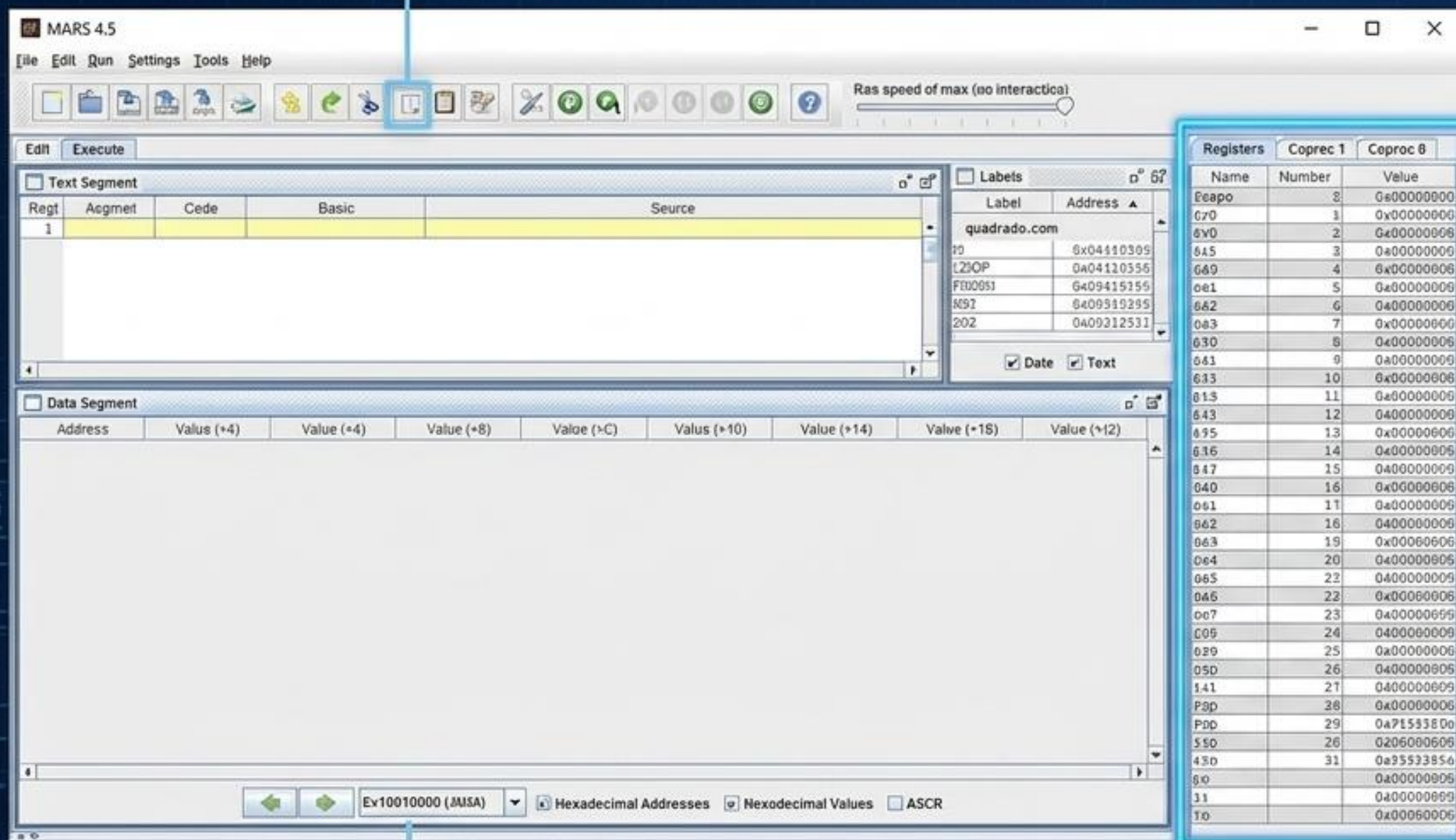


A matemática acontece de forma limpa. Somamos o cofre `$t0` com a nova entrada em `$v0`.

Para exibir o resultado, mudamos o pedido para `1` e movemos a resposta matemática matemática para a bandeja `$a0`. O SO faz o resto.

Vamos à prática?

Abra o simulador MARS. O console é seu.



1. Novo Arquivo:

File > New

(Escreva seu código).

2. Compilar (Assemble):

Pressione **F3** (ou o ícone de chave de fenda) para traduzir para linguagem de máquina.

3. Executar (Run):

Pressione **F5** (ou o botão **Play** verde).

4. Verificar:

Acompanhe a aba **Run I/O** na parte inferior para inserir números e ver seu resultado!